INFORME DE ARBITRAJE
DICTAMEN: REVISIÓN MAYOR

El manuscrito aborda un problema crítico en la adopción de arquitecturas orientadas a la nube para bases de datos relacionales: la resiliencia y el comportamiento multicapa de PostgreSQL gestionado mediante operadores en Kubernetes. Se valora positivamente el esfuerzo metodológico al aislar el dominio de fallo en un entorno productivo y el diseño de un verificador de transacciones propio (tx-verifier) capaz de rastrear de forma transparente la pérdida de *commits*.

No obstante, el estudio presenta debilidades metodológicas severas que invalidan sus principales conclusiones empíricas, carencias estructurales en su formulación teórica y afirmaciones arquitectónicas inexactas sobre el ecosistema de Kubernetes.

1. Invalidez Empírica por Artefacto de Inyección (Escenario F2)

El hallazgo central del artículo asevera que la indisponibilidad sostenida del primario (F2) no dispara un *failover*, argumentando que el operador CloudNativePG decide "esperar" basado en la persistencia del pod en el plano de control. Esta conclusión es metodológicamente incorrecta.

- La herramienta Chaos Mesh ejecuta el escenario pod-failure alterando la especificación del pod para inyectar una imagen de pausa (pause).
- CloudNativePG, al ser un operador declarativo, detecta esta discrepancia en la imagen y ejecuta automáticamente una actualización continua (*rolling update*), reiniciando el pod en su sitio.
- Por lo tanto, los 36.75 segundos reportados miden el tiempo de un reinicio de actualización forzado por la herramienta de inyección, no el tiempo de reacción ante una indisponibilidad genuina. El comportamiento documentado del operador confirma que un fallo sostenido del contenedor sí debe disparar la conmutación por error.

2. Aislamiento Ineficaz en Particiones de Red (Escenario F3)

El escenario de partición de red concluye que el operador privilegia la consistencia (comportamiento CP) al no promover una réplica y evitar el *split-brain*. Sin embargo, el diseño del experimento no logró un aislamiento efectivo.

- Las políticas de red aplicadas bloquearon la comunicación entre pods, pero el mecanismo *failsafe* de Calico permitió que las sondas de salud del kubelet siguieran operando normalmente a través del tráfico del host.
- En consecuencia, el primario nunca fue reportado como inalcanzable ante el plano de control; no hubo una "decisión" de consistencia por parte del sistema, sino una ausencia total de detección del fallo. Trabajos independientes han demostrado que, bajo un aislamiento completo, CloudNativePG es susceptible a escenarios de *split-brain* y pérdida de escrituras.

3. Vacuidad del Modelo Formal

El marco de análisis $S=(O,K,M,D)$ y sus respectivas ecuaciones (2 a 5) carecen de operatividad matemática y predictiva.

- Presentar dimensiones como funciones abstractas $f()$ sin especificar las relaciones numéricas o lógicas no aporta contenido verificable.

- El predicado de visibilidad $V(fallo, K)$, presentado como la principal contribución conceptual, es una construcción *ad hoc* ajustada exclusivamente a los tres escenarios del piloto. Dicho predicado predice erróneamente que un fallo a nivel de nodo físico no generaría promoción (puesto que el pod persiste en el registro de Kubernetes), lo cual contradice frontalmente la mecánica de expulsión de pods (node-monitor-grace-period) y el diseño del operador.

4. Imprecisiones Arquitectónicas sobre CSI y Patroni

El documento promete un análisis multicapa que incluye el almacenamiento *Container Storage Interface* (CSI), pero esta capa jamás fue ejercitada.

- Al ejecutar el primario y las dos réplicas en un único nodo (nodo-lab-01), no existió migración de cargas ni reasociación de volúmenes de red (*detach/attach*). Además, debido a la arquitectura *shared-nothing* de CloudNativePG, la reasociación de volúmenes CSI no interfiere en el camino crítico del *failover*, ya que la réplica promovida utiliza su propio volumen independiente.
- La revisión del estado del arte afirma de manera inexacta que los operadores basados en Patroni (como Zalando) requieren desplegar un clúster de consenso etcd/Raft independiente. En realidad, Patroni delega el almacenamiento del estado distribuido directamente al API Server de Kubernetes mediante el uso nativo de *Endpoints* o *ConfigMaps*.

Modificaciones Requeridas para su Aprobación

Para que el manuscrito alcance el estándar de publicación, los autores deben someter la investigación a una reestructuración empírica y teórica:

1. Repetir el escenario F2 utilizando un mecanismo de inyección que no altere la especificación del pod (por ejemplo, enviando repetidamente señales SIGKILL al proceso de la base de datos o bloqueando puertos específicos) y reinterpretar los hallazgos en función de los nuevos resultados.
2. Eliminar o dotar de contenido operacional comprobable a las ecuaciones del marco analítico, transformando el predicado de visibilidad en una hipótesis sujeta a comprobación en escenarios más amplios, y no en una ley confirmada.
3. Corregir las imprecisiones arquitectónicas respecto al rol del almacenamiento CSI en CloudNativePG y la dependencia de Patroni sobre almacenes de consenso dedicados.
4. Ajustar la precisión numérica reportada. Un RTO expresado en centésimas de segundo (por ejemplo, 7.91 s) es una métrica con precisión espuria cuando la granularidad del instrumento de recolección (tx-verifier) está limitada a 0.2 s. Las métricas deben reflejar el margen de error real del muestreo.